# SentinelSSH

Analyseur de logs SSH — Sécurité défensive

## Dossier de conception complet

Front React · API REST Flask · Back-end Python · SQLite

**Anil AKER**
Holberton School Bordeaux
Projet de fin d'année — Demo Day / Titre RNCP niveau 5
github.com/anil-akr
Version 3.1 — 1er octobre 2026

> **Le projet en une phrase.** L'administrateur envoie son fichier auth.log dans une application web ; SentinelSSH repère les adresses IP suspectes, leur donne un niveau de gravité, enregistre le résultat en base de données et l'affiche en tableau et en graphique, avec un historique des analyses.

# Historique des versions

| Version | Date | Changements |
|---|---|---|
| 1.0 | 29/09/2026 | Conception du MVP avec un front en HTML/CSS simple. |
| 2.0 | 29/09/2026 | Front en React. Ajout des user stories, des maquettes, des diagrammes de séquence, du détail des fonctions et des composants, de la spécification complète de l'API, des stratégies Git et qualité, de l'installation et de la feuille de route. |
| 3.0 | 30/09/2026 | Alignement sur le référentiel du titre RNCP 37674 (§2.5) : connexion par un compte administrateur, page de confidentialité (RGPD), interface responsive, accessibilité et éco-conception, mise en production documentée. Précisions sur le dataset Loghub et sur la lecture des dates. |
| 3.1 | 01/10/2026 | Alignement sur le Stage 3 : back-end organisé en classes (LogParser, Detector, Database, Event, SuspectIP), branche develop, pycodestyle et Postman dans la stratégie de tests, environnement de pré-production. |

Ce document est la référence du projet. Si tu t'en écartes pendant le développement, ajoute une ligne ici avec la raison : un jury apprécie une conception qui évolue de façon tracée.

# Sommaire

1. Le projet en bref 4
2. Périmètre : MVP et évolutions 4
3. User stories 7
4. Maquettes 8
5. Stack technique et justifications 10
6. Architecture 12
7. Back-end : classes et modules 14
8. Front-end React 14
9. Base de données 17
10. API REST : spécification 18
11. Diagrammes de séquence 20
12. Logique de détection 21
13. Sécurité et conformité 22
14. Stratégie de tests et qualité 23
15. Gestion du code source (Git) 24
16. Installation et lancement 25
17. Objectifs SMART 26
18. Risques et parades 27
19. Planning sur 4 semaines 27
20. Préparer la soutenance 28
Annexe — Lexique 30

# 1. Le projet en bref

## 1.1 Le problème

Un serveur Linux accessible en SSH reçoit en permanence des tentatives de connexion automatiques. Beaucoup sont des **attaques par force brute** : un robot essaie des milliers de mots de passe à la suite. Tout est écrit dans le fichier `auth.log`, mais ce fichier fait vite des milliers de lignes : une attaque peut passer inaperçue.

## 1.2 La solution

SentinelSSH est une application web. L'administrateur y envoie son fichier `auth.log`. L'outil repère les adresses IP suspectes, leur attribue un niveau de gravité, enregistre le résultat dans une base de données et l'affiche sous forme de tableau et de graphique. Il garde un historique des analyses et signale les IP qui reviennent d'une analyse à l'autre (les « récidivistes »).

## 1.3 Public cible

Administrateurs système et analystes sécurité juniors qui gèrent un serveur Linux (Ubuntu) exposé en SSH. Ils veulent une vue claire de ce qui attaque leur serveur, sans déployer un SIEM (outil professionnel qui centralise et analyse les journaux de toute une entreprise, lourd à installer).

## 1.4 Pourquoi pas simplement fail2ban ?

fail2ban est l'outil de référence contre la force brute SSH. Le jury posera la question : voici la réponse.

| Critère | fail2ban | SentinelSSH |
|---|---|---|
| Fonctionnement | Surveille les logs en continu et bloque automatiquement | Analyse un fichier à la demande |
| Interface | Fichiers de configuration, ligne de commande | Application web avec tableau, graphique et historique |
| Succès après échecs | Non signalé | Alerte **critique** : compromission possible |
| Récidives | Rebannit les récidivistes, sans vue d'ensemble à consulter | Liste consultable des IP vues dans plusieurs analyses |
| Décision de blocage | Automatique | Humaine : l'outil propose la commande, l'admin décide |

> **À retenir pour l'oral :** SentinelSSH ne remplace pas fail2ban, il le complète. fail2ban *agit*, SentinelSSH aide à *comprendre* ce qui s'est passé.

# 2. Périmètre : MVP et évolutions

Ce dossier décrit **tout le projet** : la version à construire pour le demo day (le MVP) et les versions suivantes. Le **MVP** (Minimum Viable Product) est la plus petite version du produit qui fonctionne vraiment et qu'on peut montrer.

## 2.1 Fonctionnalités du MVP

| # | Fonctionnalité | Ce que ça fait | Semaine |
|---|---|---|---|
| F1 | Envoi d'un fichier | Formulaire d'upload. Fichier .log ou .txt, 5 Mo maximum | S2–S3 |
| F2 | Lecture des logs | Chaque ligne utile devient un « événement ». Les autres sont ignorées et comptées | S1 |
| F3 | Détection | 3 règles, 3 niveaux de gravité (§12) | S1 |
| F4 | Enregistrement | L'analyse et ses IP suspectes sont stockées dans SQLite. Le fichier brut n'est jamais stocké | S2 |
| F5 | Résultats | Résumé chiffré + tableau des IP suspectes, triées par gravité puis par nombre d'échecs | S3 |
| F6 | Graphique | Barres : top 10 des IP par nombre d'échecs | S3 |
| F7 | Blocage suggéré | Commande iptables prête à copier pour les IP de gravité élevée ou critique. Rien n'est exécuté | S2–S3 |
| F8 | Historique | Liste des analyses passées, avec suppression | S2–S3 |
| F9 | IP récidivistes | IP suspectes présentes dans au moins 2 analyses différentes | S2–S3 |
| F10 | Mode terminal | `python cli.py fichier.log` affiche les résultats sans passer par le web | S1 |
| F11 | Connexion | Un compte administrateur unique ; mot de passe haché ; toutes les données sont protégées par une session | S2–S3 |
| F12 | Confidentialité | Page qui explique quelles données sont traitées, pourquoi, combien de temps, et les droits (RGPD) | S3 |
| F13 | Responsive | Interface utilisable sur ordinateur et sur mobile | S3 |

## 2.2 Bonus (seulement si tu es en avance en semaine 4)

- Export CSV des résultats d'une analyse.
- Filtre du tableau par niveau de gravité.
- Limitation des tentatives de connexion : 1 minute d'attente après 5 échecs (un clin d'œil au sujet du projet).

## 2.3 Hors périmètre du MVP, et pourquoi

- **Blocage automatique des IP :** l'application aurait besoin des droits root, un risque trop grand pour un MVP.
- **Surveillance en temps réel :** lire les logs en continu demande une architecture plus complexe.
- **Plusieurs comptes et gestion des rôles :** un seul compte administrateur suffit pour le MVP.
- **Autres sources :** journald, autres formats de logs, autres protocoles que SSH.
- **Géolocalisation des IP** et **déploiement sur Internet**.

## 2.4 Feuille de route (après le demo day)

| Version | Contenu | Ce que ça apporte |
|---|---|---|
| **V1 — MVP** demo day | Tout le §2.1 | Un outil complet et démontrable : front, API, back, base de données |
| **V2** | Plusieurs comptes et rôles (administrateur, lecteur) · lecture de journald et du format de date ISO · export CSV/PDF · alertes par e-mail | Utilisable sur un vrai serveur, par une équipe |
| **V3** | Agent qui lit les logs en temps réel · plusieurs serveurs · blocage validé par l'admin puis appliqué par un service séparé aux droits limités | Passage d'un outil d'analyse à un outil de surveillance |
| **Plus tard** | Honeypot + intégration à un SIEM (piste pour le RNCP 6) | Attirer et étudier les attaquants, centraliser les alertes |

Cette feuille de route répond à la question du jury « et après ? » : le MVP est petit, mais il s'inscrit dans une vision.

## 2.5 Couverture du référentiel RNCP

Le titre RNCP niveau 5 préparé à Holberton est le titre professionnel **« Développeur web et web mobile »** (RNCP 37674). Son référentiel contient 8 compétences, réparties en 2 blocs. Voici où chacune apparaît dans le projet.

| Bloc | Compétence du référentiel | Où dans SentinelSSH |
|---|---|---|
| **Front-end sécurisé** | Installer et configurer son environnement de travail | Node.js, Vite, ESLint, environnement virtuel Python, Git (§16) |
| | Maquetter des interfaces utilisateur | 4 maquettes : accueil, détail, connexion, mobile (§4) |
| | Réaliser des interfaces utilisateur statiques | HTML/CSS responsive, accessibilité, éco-conception (§8.5) |
| | Développer la partie dynamique des interfaces | React : composants, state, appels à l'API, routes protégées (§8) |
| **Back-end sécurisé** | Mettre en place une base de données relationnelle | SQLite : 3 tables, clé étrangère, contraintes (§9) |
| | Développer des composants d'accès aux données SQL et NoSQL | SQL : `database.py`, requêtes paramétrées (§9). **NoSQL : non couvert** (voir ci-dessous) |
| | Développer des composants métier côté serveur | Lecture des logs, détection, API REST, authentification (§7, §10, §12) |
| | Documenter le déploiement d'une application dynamique | Mise en production locale documentée (§16.2) |
| **Transversal** | Sécurité, RGPD, tests | Sécurité et page de confidentialité (§13), stratégie de tests (§14) |

> **À vérifier avec ton équipe pédagogique :** le référentiel cite « SQL et NoSQL ». Le projet couvre le SQL. Demande si le NoSQL doit apparaître dans ton projet ou s'il est évalué autrement. S'il doit y être, la piste la plus simple est un **journal d'audit** (qui s'est connecté, qui a lancé ou supprimé une analyse) rangé dans une base orientée documents. C'est utile pour la sécurité et cohérent avec ton objectif GRC.

# 3. User stories

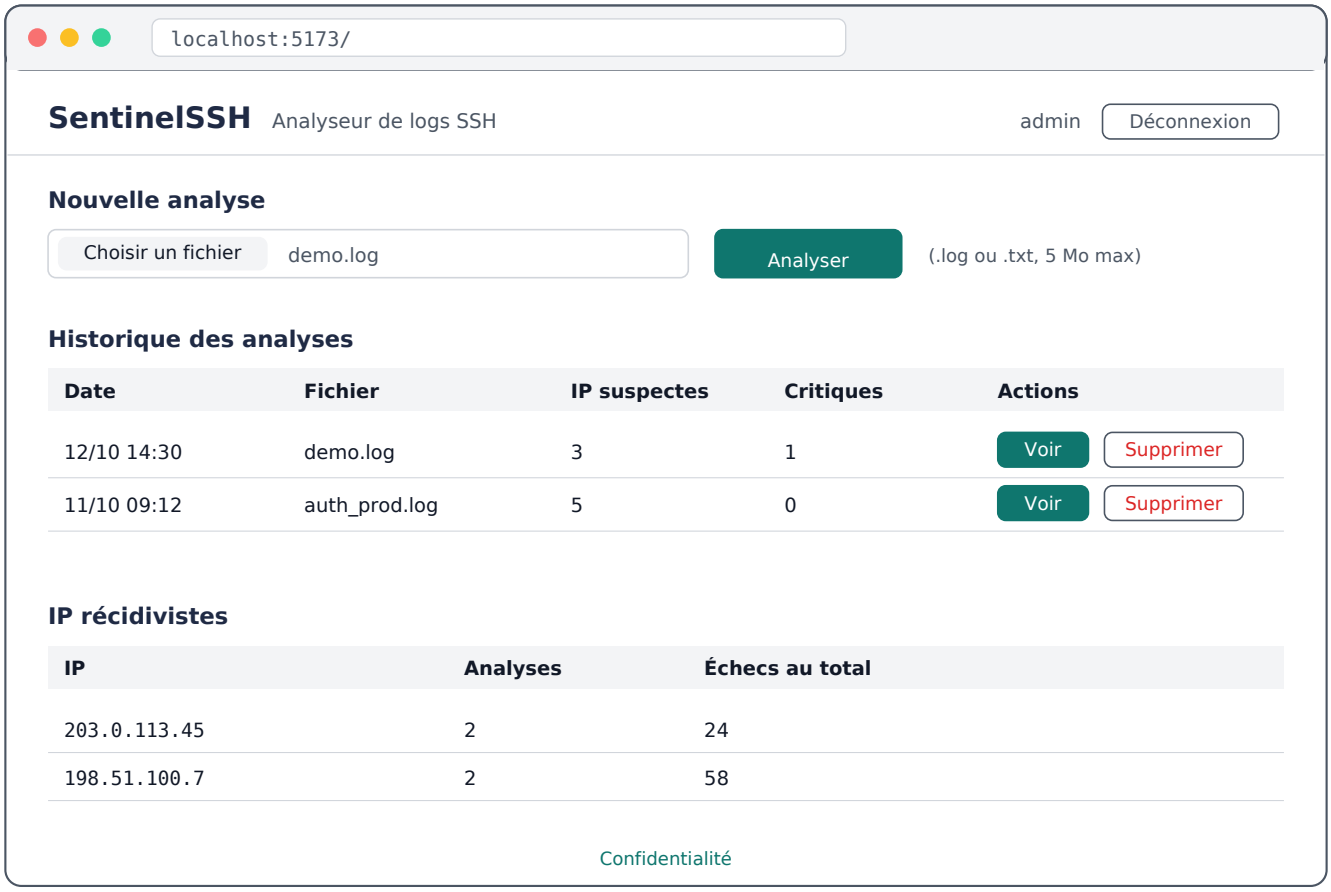
Une **user story** décrit un besoin du point de vue de l'utilisateur : « En tant que…, je veux…, afin de… ». La priorité suit la méthode **MoSCoW** : *Must* (indispensable), *Should* (important), *Could* (bonus), *Won't* (pas dans cette version). Les critères d'acceptation disent quand la story est terminée.

| ID | User story | Critères d'acceptation | Priorité |
|---|---|---|---|
| US0 | En tant qu'admin, je veux me connecter avec un compte protégé par mot de passe afin que personne d'autre ne voie mes analyses. | Page de connexion · sans connexion, toute page protégée renvoie vers /login · bouton « Déconnexion » | Must |
| US1 | En tant qu'admin, je veux envoyer mon fichier auth.log afin d'obtenir une analyse sans lire les lignes à la main. | Fichier .log/.txt de 5 Mo max accepté · message clair sinon · redirection vers le résultat | Must |
| US2 | En tant qu'admin, je veux voir les IP suspectes classées par gravité afin de traiter d'abord les plus dangereuses. | Tableau trié : critique, puis élevée, puis moyenne, puis par nombre d'échecs | Must |
| US3 | En tant qu'admin, je veux être alerté quand une IP réussit à se connecter après des échecs afin de vérifier une compromission. | Gravité « critique » + règle R3 affichée · compteur d'alertes critiques dans le résumé | Must |
| US4 | En tant qu'admin, je veux copier la commande de blocage d'une IP afin de la bloquer moi-même. | Commande iptables (ou ip6tables) affichée · bouton « Copier » avec retour « Copié ! » | Must |
| US5 | En tant qu'admin, je veux retrouver mes analyses passées afin de suivre l'évolution des attaques. | Historique trié du plus récent au plus ancien · accès au détail en un clic | Must |
| US6 | En tant qu'admin, je veux voir un graphique des IP les plus actives afin de comprendre la situation d'un coup d'œil. | Barres du top 10 des IP par nombre d'échecs | Should |
| US7 | En tant qu'admin, je veux voir les IP qui reviennent dans plusieurs analyses afin de repérer les attaquants persistants. | Liste des IP présentes dans au moins 2 analyses, avec le total d'échecs | Should |
| US8 | En tant qu'admin, je veux supprimer une analyse afin de ne pas garder de données inutiles (RGPD). | Confirmation avant suppression · l'analyse et ses IP disparaissent | Should |
| US9 | En tant qu'admin, je veux utiliser l'application sur mon téléphone afin de vérifier une alerte en déplacement. | Toutes les pages lisibles sur un écran de téléphone, sans défilement horizontal de la page | Should |
| US10 | En tant qu'utilisateur, je veux savoir quelles données sont conservées et combien de temps afin de connaître mes droits. | Page « Confidentialité » accessible depuis toutes les pages, même sans connexion | Should |
| US11 | En tant qu'admin, je veux exporter une analyse en CSV afin de l'archiver ou la partager. | Fichier CSV téléchargé avec les colonnes du tableau | Could |
| US12 | En tant qu'admin, je veux que l'outil bloque les IP tout seul. | Hors MVP : voir §2.3 et §2.4 | Won't |

# 4. Maquettes

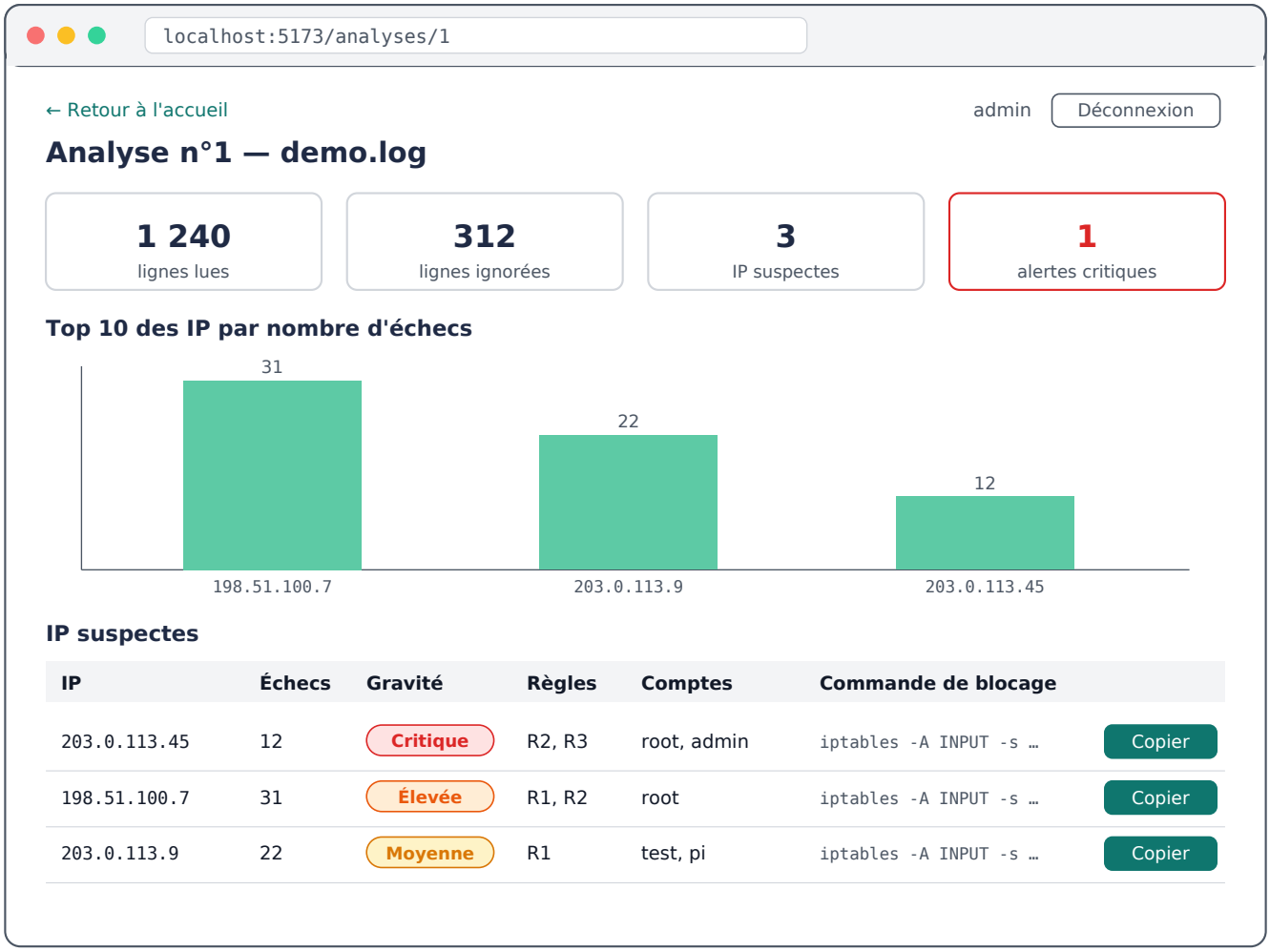
Les maquettes (ou *wireframes*) montrent l'organisation des écrans, pas le design final. L'application a 4 pages : connexion, accueil, détail d'une analyse et confidentialité (texte simple, sans maquette).

## 4.1 Page d'accueil — /



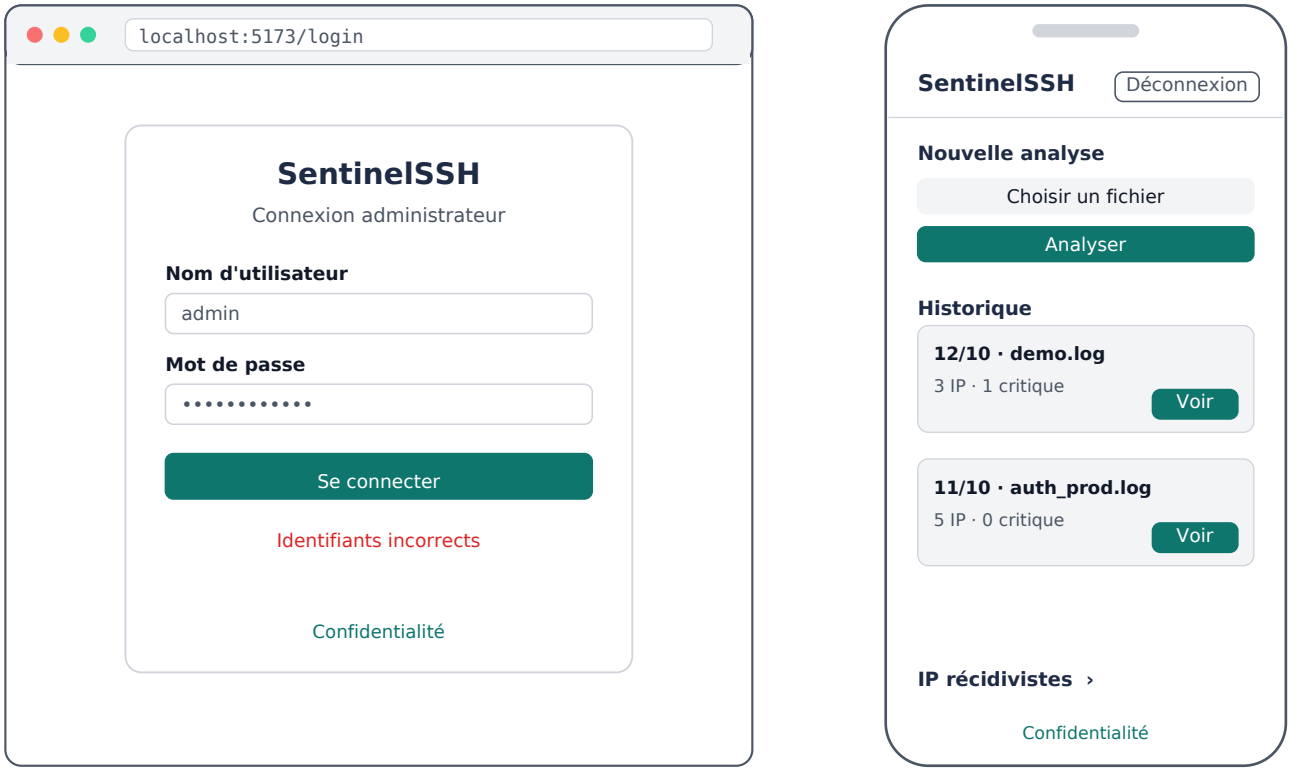

*Envoi d'un fichier, historique des analyses et IP récidivistes sur la même page.*

## 4.2 Page de détail — /analyses/:id




*Résumé en 4 chiffres, graphique, puis tableau des IP suspectes avec la gravité en couleur et en texte.*

### 4.3 Connexion et affichage mobile




Connexion (ordinateur) Accueil (mobile)

*À gauche, la page de connexion avec son message d'erreur. À droite, l'accueil sur téléphone : les tableaux deviennent des cartes empilées.*

# 5. Stack technique et justifications

| Couche | Techno | Rôle |
|---|---|---|
| Front-end | React + Vite, React Router, Recharts | Interface : 4 pages, des composants, un graphique |
| Communication | API REST en JSON, `fetch` côté front | Échanges entre le front et le back |
| Back-end | Python 3 + Flask | API, vérification des fichiers, appel du cœur d'analyse |
| Base de données | SQLite via le module `sqlite3` intégré à Python | Compte administrateur, historique des analyses et IP suspectes |
| Sécurité | Werkzeug (fourni avec Flask), sessions Flask | Hachage des mots de passe, session de connexion par cookie signé |
| Mise en production | Gunicorn | Serveur de production qui lance l'application en une commande |
| Tests et qualité | pytest, pytest-cov, ESLint | Vérifier le back, mesurer la couverture, contrôler le code du front |
| Outils | Git, GitHub, GitHub Projects, Node.js LTS + npm | Versions du code, suivi des tâches, outillage du front |

## Pourquoi ces choix plutôt que d'autres

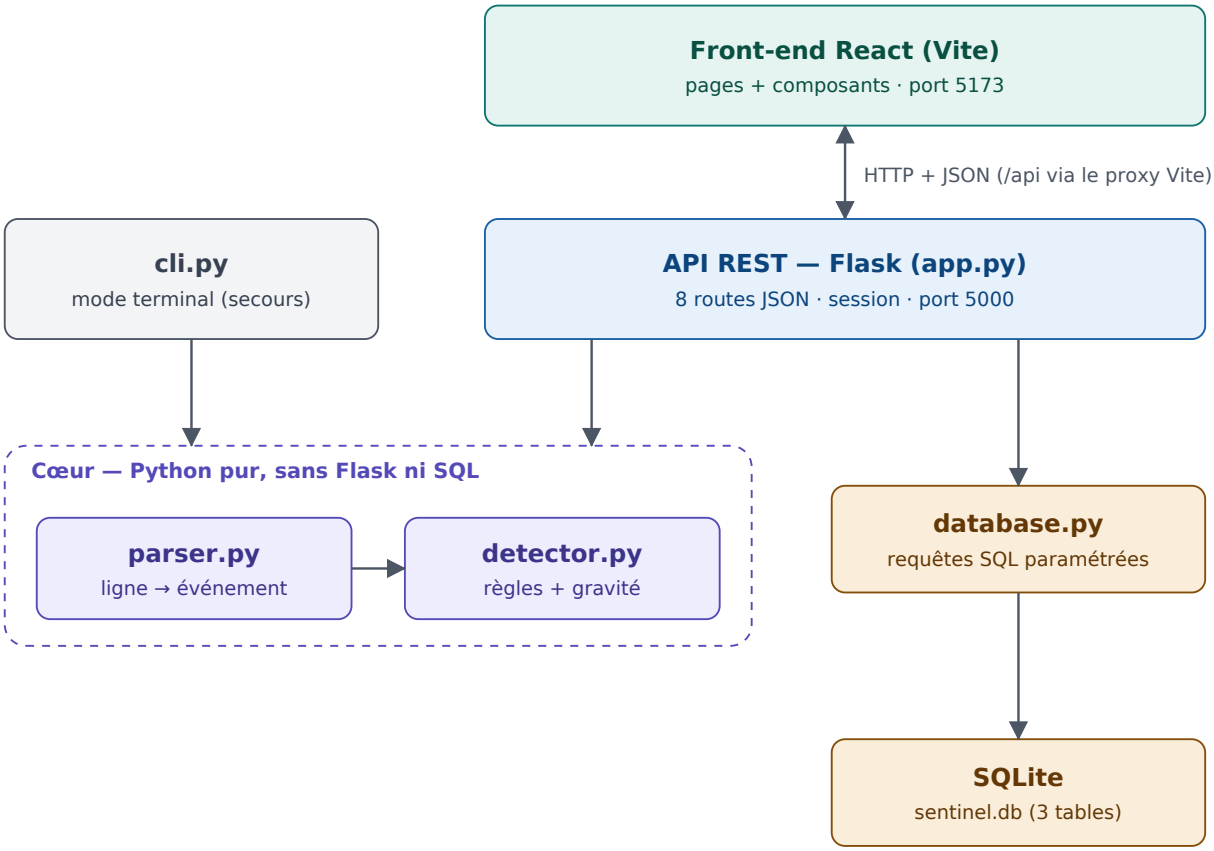
| Besoin | Choix | Alternative écartée | Pourquoi |
|---|---|---|---|
| Framework back | Flask | Django, FastAPI | Django est trop lourd ici (admin, ORM, templates). FastAPI génère une documentation automatique, mais ajoute des notions (typage, async, Pydantic). |
| Front | React + Vite | Vue, Angular, Create React App | React est le standard le plus demandé. Vite démarre un projet en une commande ; Create React App est abandonné. |
| Base | SQLite | PostgreSQL, MongoDB | Outil local, un seul utilisateur : pas besoin d'un serveur de base. Les données sont reliées (1 → N) : une base SQL est adaptée. |
| Accès à la base | SQL écrit à la main | ORM (SQLAlchemy) | Tu comprends chaque requête et tu montres comment tu te protèges des injections SQL. |
| Graphique | Recharts | Chart.js | Recharts s'utilise directement avec des composants React. |
| Appels API | fetch | axios | Intégré au navigateur : aucune dépendance en plus. |
| Style | CSS simple | Tailwind, Bootstrap | Une notion de moins à apprendre en 1 mois. |
| Connexion | Session Flask (cookie signé) | Jeton JWT | Plus simple : rien à stocker côté JavaScript, et le cookie HttpOnly est inaccessible à un script malveillant. |
| Serveur | Gunicorn | Serveur de développement Flask | Le serveur intégré à Flask n'est pas conçu pour la production. |

> **Pourquoi garder Python côté serveur ?** Le cœur du projet, c'est la détection : c'est là que viendront les questions techniques, et Python est ton langage le plus solide. React couvre la partie JavaScript.

# 6. Architecture

## 6.1 Vue d'ensemble

Le projet suit une architecture en 3 couches : le **front-end** React (ce que voit l'utilisateur), l'**API** Flask (la porte d'entrée du serveur) et le **back-end** (le cœur d'analyse et la base de données).




*Les flèches montrent qui appelle qui.* `app.py` *orchestre : il appelle le cœur, puis* `database.py`*.*

> **Le principe clé :** `parser.py` et `detector.py` ne connaissent ni Flask ni SQL. Ils reçoivent du texte et renvoient des résultats. Tu peux donc les tester sans serveur ni base, et `cli.py` les réutilise tels quels. Si le web plante le jour J, la démo tourne en ligne de commande.

## 6.2 Parcours d'une analyse, étape par étape

**Avant tout :** l'utilisateur se connecte sur `/login`. L'API vérifie le mot de passe et ouvre une session : le navigateur reçoit un cookie qu'il renverra automatiquement à chaque appel suivant (§11.1).

1. L'utilisateur choisit un fichier dans `UploadForm` et clique sur « Analyser ».
2. `api.js` envoie le fichier à `POST /api/analyses` avec `fetch` et un `FormData`.
3. `app.py` vérifie la session (401 sinon), puis le fichier : présence, extension, taille (400 ou 413 en cas de problème).
4. `parser.py` transforme chaque ligne utile en événement.
5. `detector.py` applique les 3 règles et calcule la gravité de chaque IP.
6. `database.py` enregistre l'analyse puis ses IP suspectes dans SQLite.
7. L'API répond 201 avec l'identifiant de l'analyse. React affiche la page `/analyses/<id>`.

8. `AnalysisPage` appelle `GET /api/analyses/<id>` et affiche le résumé, le graphique et le tableau.

## 6.3 Arborescence du projet

```
sentinelssh/
├── backend/
│   ├── app.py                  # Flask : routes /api, connexion, erreurs,
│   │                           # commande create-admin, sert le front en production
│   ├── cli.py                  # mode terminal (n'utilise pas la base)
│   ├── schema.sql              # création des 3 tables
│   ├── requirements.txt        # flask, python-dotenv, gunicorn, pytest, pytest-cov, pycodestyle
│   ├── .env.example            # modèle : SECRET_KEY=... (le vrai .env n'est jamais commité)
│   ├── sentinel/
│   │   ├── __init__.py         # vide : fait de "sentinel" un package
│   │   ├── config.py           # seuils, chemin de la base, taille max, SECRET_KEY
│   │   ├── models.py           # dataclasses Event et SuspectIP
│   │   ├── parser.py           # classe LogParser : ligne de log -> Event (ou None)
│   │   ├── detector.py         # classe Detector : Event -> SuspectIP + gravité
│   │   └── database.py         # classe Database : toutes les requêtes SQL, et seulement elles
│   └── tests/
│       ├── samples/            # mini-logs dont tu connais le résultat
│       ├── test_parser.py
│       ├── test_detector.py
│       └── test_api.py
├── frontend/                   # créé avec : npm create vite@latest
│   ├── package.json
│   ├── vite.config.js          # proxy /api -> Flask
│   └── src/
│       ├── main.jsx            # point d'entrée React
│       ├── App.jsx             # déclaration des 4 routes
│       ├── api.js              # toutes les fonctions qui appellent l'API
│       ├── styles.css
│       ├── pages/              # LoginPage, HomePage, AnalysisPage, PrivacyPage
│       └── components/         # ProtectedRoute, Header, UploadForm, HistoryTable,
│                               # RecidivistsTable, SummaryCards, TopIpsChart,
│                               # SuspectsTable, SeverityBadge, CopyButton
├── demo/demo.log               # le log qui « raconte l'histoire » au jury
├── .github/workflows/tests.yml # CI : lance pytest à chaque push
├── .gitignore
└── README.md
```

## 6.4 Comment le front parle au back : le proxy Vite

En développement, React tourne sur le port 5173 (Vite) et Flask sur le port 5000. Pour le navigateur, un port différent = une autre « origine ». Par sécurité, il bloque par défaut les requêtes vers une autre origine : c'est la règle **CORS** (Cross-Origin Resource Sharing).

La solution la plus simple est le **proxy de Vite** : le navigateur ne parle qu'à Vite, et Vite transmet tout ce qui commence par `/api` à Flask. Pour le navigateur, tout vient de la même origine : pas besoin d'ouvrir CORS côté Flask.

```
// frontend/vite.config.js
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      '/api': 'http://127.0.0.1:5000', // les appels /api partent vers Flask
    },
  },
})
```

Côté React, on écrit donc simplement `fetch('/api/analyses')`, sans adresse de serveur. Le cookie de session passe lui aussi par le proxy. En production (§16.2), Flask sert directement le front construit : tout est sur la même origine et le proxy n'est plus utile.

# 7. Back-end : classes et modules

Le back-end est organisé en 5 classes, chacune avec une seule responsabilité (diagramme de classes : Stage 3, §3.1). Les noms ci-dessous sont un plan : tu écris le contenu toi-même.

| Élément | Attributs | Méthodes et rôle |
|---|---|---|
| `config.py` | Constantes | Seuils des règles (`FAST_THRESHOLD = 5`, `FAST_WINDOW = 60`, `PERSISTENT_THRESHOLD = 20`, `COMPROMISE_THRESHOLD = 5`), `DB_PATH`, `MAX_UPLOAD_BYTES`, `ALLOWED_EXTENSIONS`, `SECRET_KEY` (lue dans l'environnement) |
| `Event` (dataclass) | time, kind, user, ip | Une tentative de connexion lue dans le journal |
| `SuspectIP` (dataclass) | ip, failures, severity, rules, users, first_seen, last_seen | `to_dict()` : conversion pour le JSON et la base |
| `LogParser` | year (2000, bissextile), expressions régulières | `parse_line(line)` : un `Event` ou `None` · `parse_text(text)` : événements, lignes lues, lignes ignorées |
| `Detector` | Les 4 seuils | `detect(events)` : liste de `SuspectIP` triée · `_is_persistent`, `_is_fast`, `_is_compromised` (R1, R2, R3) · `severity_of(rules)` |
| `Database` | db_path | `connect()` (active les clés étrangères) · `init_db()` · `save_analysis`, `list_analyses`, `get_analysis`, `delete_analysis`, `get_recidivists` · `get_user`, `create_user` |
| `app.py` (module) | Application Flask | 8 routes · décorateur `login_required` (401 sans session) · `block_command(ip)` (iptables ou ip6tables) · erreurs 400, 401, 404, 413 · commande `create-admin` (mot de passe de 12 caractères minimum) · service de `frontend/dist` en production |
| `cli.py` (module) | — | `main()` : analyse un fichier passé en argument, affiche un tableau texte, sans base |

Une **dataclass** est une classe qui sert surtout à ranger des données : Python écrit lui-même son constructeur. Les méthodes qui commencent par _ sont privées par convention : elles ne servent qu'à l'intérieur de la classe.

# 8. Front-end React

## 8.1 Arbre des composants

Un **composant** est une fonction qui renvoie un morceau d'interface. Les composants s'emboîtent :

```
App                          (déclare les routes)
├── LoginPage                "/login"            (publique)
├── PrivacyPage              "/confidentialite"  (publique)
└── ProtectedRoute           (vérifie la session, sinon renvoie vers /login)
    ├── Header               (nom de l'admin + bouton Déconnexion)
    ├── HomePage             "/"
    │   ├── UploadForm
    │   ├── HistoryTable
    │   └── RecidivistsTable
    └── AnalysisPage         "/analyses/:id"
        ├── SummaryCards
        ├── TopIpsChart
        └── SuspectsTable
            ├── SeverityBadge
            └── CopyButton
```

## 8.2 Rôle de chaque composant

Les **props** sont les données qu'un composant reçoit de son parent. Le **state** est ce qu'il garde lui-même : quand le state change, React réaffiche le composant.

| Composant | Rôle | Reçoit (props) | Garde (state) |
|---|---|---|---|
| `LoginPage` | Formulaire nom + mot de passe ; appelle `login()` puis ouvre l'accueil ; affiche « Identifiants incorrects » en cas d'échec | — | username, password, error |
| `PrivacyPage` | Texte fixe : données traitées, finalité, durée de conservation, droits (RGPD) | — | — |
| `ProtectedRoute` | Appelle `me()` au chargement : affiche les pages protégées si la session est valide, sinon redirige vers /login | — | status |
| `Header` | Titre, nom de l'admin, bouton « Déconnexion » (`logout()`) | username | — |
| `HomePage` | Charge l'historique et les récidivistes au chargement (`useEffect`). Recharge les deux après une suppression | — | analyses, recidivists, error |
| `UploadForm` | Choix du fichier et envoi, puis ouverture de la page de détail (`useNavigate`) | — | file, loading, error |
| `HistoryTable` | Tableau des analyses, boutons Voir / Supprimer (avec confirmation) | analyses, onDelete | — |
| `RecidivistsTable` | Tableau des IP récidivistes | recidivists | — |
| `AnalysisPage` | Lit l'id dans l'URL (`useParams`) et charge l'analyse | — | analysis, loading, error |
| `SummaryCards` | Les 4 chiffres du résumé | analysis | — |
| `TopIpsChart` | Graphique en barres du top 10 (Recharts) | suspects | — |
| `SuspectsTable` | Tableau des IP suspectes | suspects | — |
| `SeverityBadge` | Pastille colorée + texte de la gravité | severity | — |
| `CopyButton` | Copie un texte (`navigator.clipboard`) et affiche « Copié ! » | text | copied |

## 8.3 Le fichier api.js

Toutes les requêtes vers l'API sont rangées dans un seul fichier. Les composants n'appellent jamais `fetch` directement : si une route change, tu ne modifies qu'un endroit.

- `login(username, password)`, `logout()`, `me()`, `uploadLog(file)`, `listAnalyses()`, `getAnalysis(id)`, `deleteAnalysis(id)`, `listRecidivists()`.
- Si l'API répond 401 (session expirée), la fonction renvoie l'utilisateur vers `/login`.
- Chaque fonction renvoie les données JSON, ou lance une erreur contenant le message `"error"` renvoyé par l'API. Le composant attrape l'erreur et l'affiche à l'utilisateur.

## 8.4 États à gérer dans chaque page

- **Chargement :** afficher « Chargement… » pendant l'appel à l'API.
- **Erreur :** afficher le message (API arrêtée, analyse introuvable, fichier refusé).
- **Vide :** « Aucune analyse pour l'instant » ou « Aucune IP suspecte dans ce fichier ».

> **Sécurité du front :** React échappe automatiquement tout ce que tu affiches avec des accolades `{...}` dans le JSX : un nom d'utilisateur piégé s'affiche comme du texte, sans s'exécuter. La seule façon de casser cette protection est `dangerouslySetInnerHTML` : ne l'utilise jamais.

## 8.5 Responsive, accessibilité et éco-conception

- **Responsive :** mise en page en flexbox ou grid ; sur un petit écran, les tableaux défilent horizontalement dans leur cadre (ou deviennent des cartes), sans que la page entière déborde. Vérification avec le mode responsive des outils du navigateur.
- **Accessibilité :** chaque champ de formulaire a un `<label>` ; les boutons ont un texte clair ; la gravité est affichée en couleur *et* en texte ; contrastes suffisants ; tout est utilisable au clavier.
- **Éco-conception :** peu de dépendances, aucune image lourde, données chargées seulement quand une page en a besoin, graphique limité au top 10.

# 9. Base de données

La base contient 3 tables. `analyses` et `suspect_ips` sont reliées par une relation « 1 → N » : une analyse contient zéro, une ou plusieurs IP suspectes. `users` contient le compte administrateur.

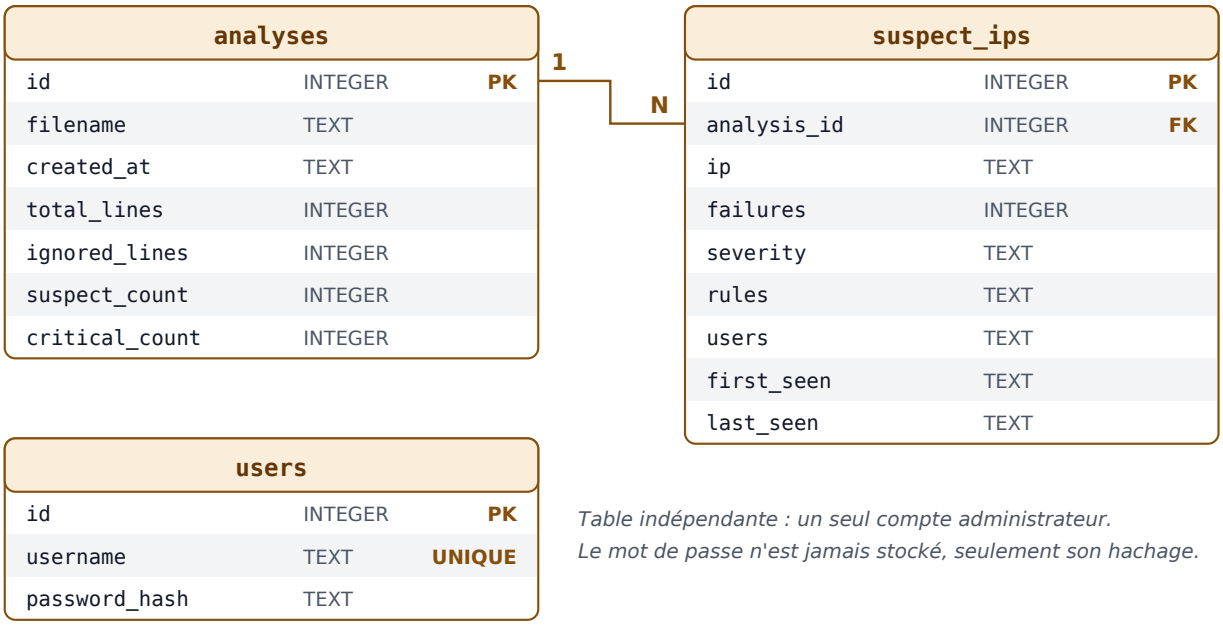



*Table indépendante : un seul compte administrateur.*
*Le mot de passe n'est jamais stocké, seulement son hachage.*

*PK = clé primaire (identifiant unique d'une ligne). FK = clé étrangère (colonne qui pointe vers la clé primaire d'une autre table).*

## 9.1 Le rôle de chaque table

- `analyses` : une ligne = une analyse, donc un fichier envoyé. Les compteurs (`total_lines`, `suspect_count`…) évitent de tout recalculer pour afficher l'historique.
- `suspect_ips` : une ligne = une IP suspecte *dans une analyse donnée*. La même IP peut apparaître dans plusieurs analyses : c'est ce qui permet de trouver les récidivistes.
- `users` : le compte administrateur. `username` est `UNIQUE`. `password_hash` contient le **hachage** du mot de passe, calculé par `generate_password_hash` de Werkzeug : une empreinte à sens unique, impossible à retransformer en mot de passe (contrairement à un chiffrement, qui se déchiffre).

## 9.2 Points importants

- **Suppression en cascade :** `analysis_id` est déclarée avec `ON DELETE CASCADE`. Supprimer une analyse supprime ses IP.
- **Piège SQLite :** les clés étrangères sont désactivées par défaut. Sans `PRAGMA foreign_keys = ON;` à chaque connexion, la cascade ne fait rien.
- **Contraintes :** `NOT NULL` sur les colonnes obligatoires, et `CHECK` pour que `severity` vaille uniquement `moyenne`, `elevee` ou `critique`.
- **Colonnes texte simples :** `rules` (ex. `R2,R3`) et `users` (ex. `root,admin,anil`) sont stockées en texte séparé par des virgules. Un modèle « parfait » utiliserait une table en plus, inutile pour le MVP. Sache l'expliquer.
- **Dates :** `created_at` au format ISO 8601 (ex. `2026-10-12T14:30:00`), qui se trie correctement comme du texte.

- **Pourquoi ne pas stocker chaque ligne du log ?** Pour le volume, et pour la minimisation des données exigée par le RGPD (§13).
- **Création :** `schema.sql` contient les `CREATE TABLE IF NOT EXISTS` et est exécuté au démarrage.
- **Git :** le fichier `sentinel.db` ne doit jamais être commité (il contient des IP) : il va dans `.gitignore`.

### 9.3 Les requêtes dont tu as besoin

1. Lire un utilisateur par son nom, pour la connexion.
2. Insérer une analyse, puis récupérer son identifiant (`cursor.lastrowid`).
3. Insérer les IP suspectes de cette analyse.
4. Lister les analyses, les plus récentes d'abord (`ORDER BY created_at DESC`).
5. Lire une analyse et ses IP, triées par gravité puis par échecs. Indice : `ORDER BY` accepte une expression `CASE`.
6. Trouver les récidivistes. Indice : `GROUP BY`, `HAVING` et `COUNT(DISTINCT ...)`.
7. Supprimer une analyse : un seul `DELETE` sur `analyses`, la cascade fait le reste.

## 10. API REST : spécification

Une **API REST** est un ensemble d'adresses (des **routes**) que le front appelle pour lire ou modifier des données. Chaque route combine une méthode HTTP (`GET` pour lire, `POST` pour créer, `DELETE` pour supprimer) et une adresse. Les données circulent en **JSON**.

### 10.1 Les routes

| Méthode | Route | Rôle | Réponses |
|---|---|---|---|
| POST | `/api/auth/login` | Vérifie le nom et le mot de passe, ouvre la session | **200** nom de l'admin + cookie<br>**400** champ manquant<br>**401** identifiants incorrects |
| POST | `/api/auth/logout` | Ferme la session | **204** |
| GET | `/api/auth/me` | Indique qui est connecté | **200** nom de l'admin<br>**401** pas de session |
| POST | `/api/analyses` | Reçoit un fichier (champ `file`), l'analyse, enregistre le résultat | **201** résumé + id<br>**400** fichier absent ou extension refusée<br>**413** fichier trop gros |
| GET | `/api/analyses` | Liste l'historique, du plus récent au plus ancien | **200** |
| GET | `/api/analyses/<id>` | Une analyse avec ses IP suspectes triées | **200**<br>**404** analyse introuvable |
| DELETE | `/api/analyses/<id>` | Supprime l'analyse et ses IP | **204** sans contenu<br>**404** analyse introuvable |
| GET | `/api/recidivists` | IP présentes dans au moins 2 analyses | **200** |

Les 5 routes `/api/analyses` et `/api/recidivists` sont protégées : sans session, elles répondent **401**. Il y a donc 8 routes en tout, dont 3 pour la connexion.

### 10.2 Conventions

- Toujours du JSON, même pour les erreurs : `{"error": "message clair"}`.

- **201** après un POST réussi (une ressource a été créée), **204** après un DELETE réussi (rien à renvoyer).
- Pour l'erreur 413, Flask renvoie une page HTML par défaut : ajoute `@app.errorhandler(413)` pour renvoyer du JSON. Fais de même pour 404.
- Un fichier valide sans aucune ligne utile donne quand même une analyse (201), avec 0 IP suspecte.
- Connexion refusée : toujours le même message (« Identifiants incorrects »), que le nom ou le mot de passe soit faux. Sinon, un attaquant pourrait deviner quels noms d'utilisateur existent.
- `block_command` est calculée par l'API, jamais stockée en base.

## 10.3 Exemples de réponses

```
POST /api/auth/login   {"username": "admin", "password": "..."}  ->  200
{"username": "admin"}                       (+ cookie de session HttpOnly)

POST /api/analyses  ->  201
{"id": 1, "filename": "demo.log", "suspect_count": 3, "critical_count": 1}

POST /api/analyses (mauvaise extension)  ->  400
{"error": "Extension non autorisée : .log ou .txt uniquement"}

GET /api/analyses  ->  200
[{"id": 2, "filename": "auth_prod.log", "created_at": "2026-10-13T09:12:00",
  "total_lines": 5210, "suspect_count": 5, "critical_count": 0},
 {"id": 1, "filename": "demo.log", "created_at": "2026-10-12T14:30:00",
  "total_lines": 1240, "suspect_count": 3, "critical_count": 1}]

GET /api/recidivists  ->  200
[{"ip": "198.51.100.7", "analyses_count": 2, "total_failures": 58}]

GET /api/analyses (sans session)  ->  401
{"error": "Connexion requise"}
```

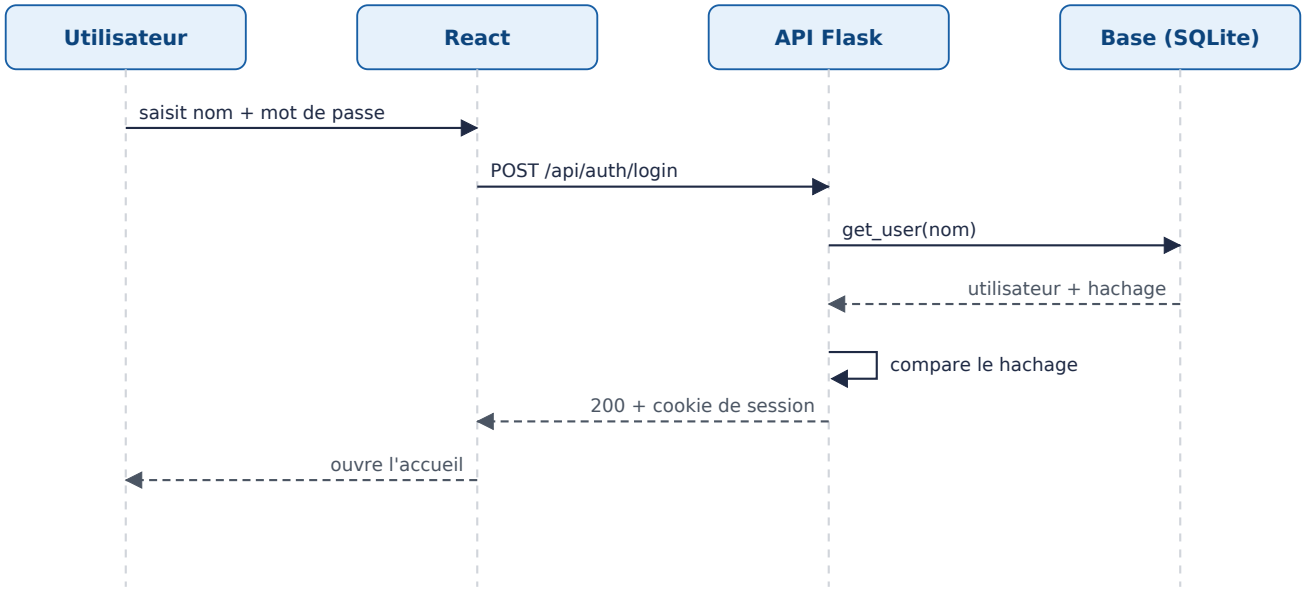
### GET /api/analyses/1 → 200

```
{
  "id": 1, "filename": "demo.log", "created_at": "2026-10-12T14:30:00",
  "total_lines": 1240, "ignored_lines": 312, "suspect_count": 3, "critical_count": 1,
  "suspects": [
    {
      "ip": "203.0.113.45", "failures": 12, "severity": "critique",
      "rules": ["R2", "R3"], "users": ["root", "admin", "anil"],
      "first_seen": "Oct 12 14:03:21", "last_seen": "Oct 12 14:05:02",
      "block_command": "iptables -A INPUT -s 203.0.113.45 -j DROP"
    }
  ]
}
```

# 11. Diagrammes de séquence

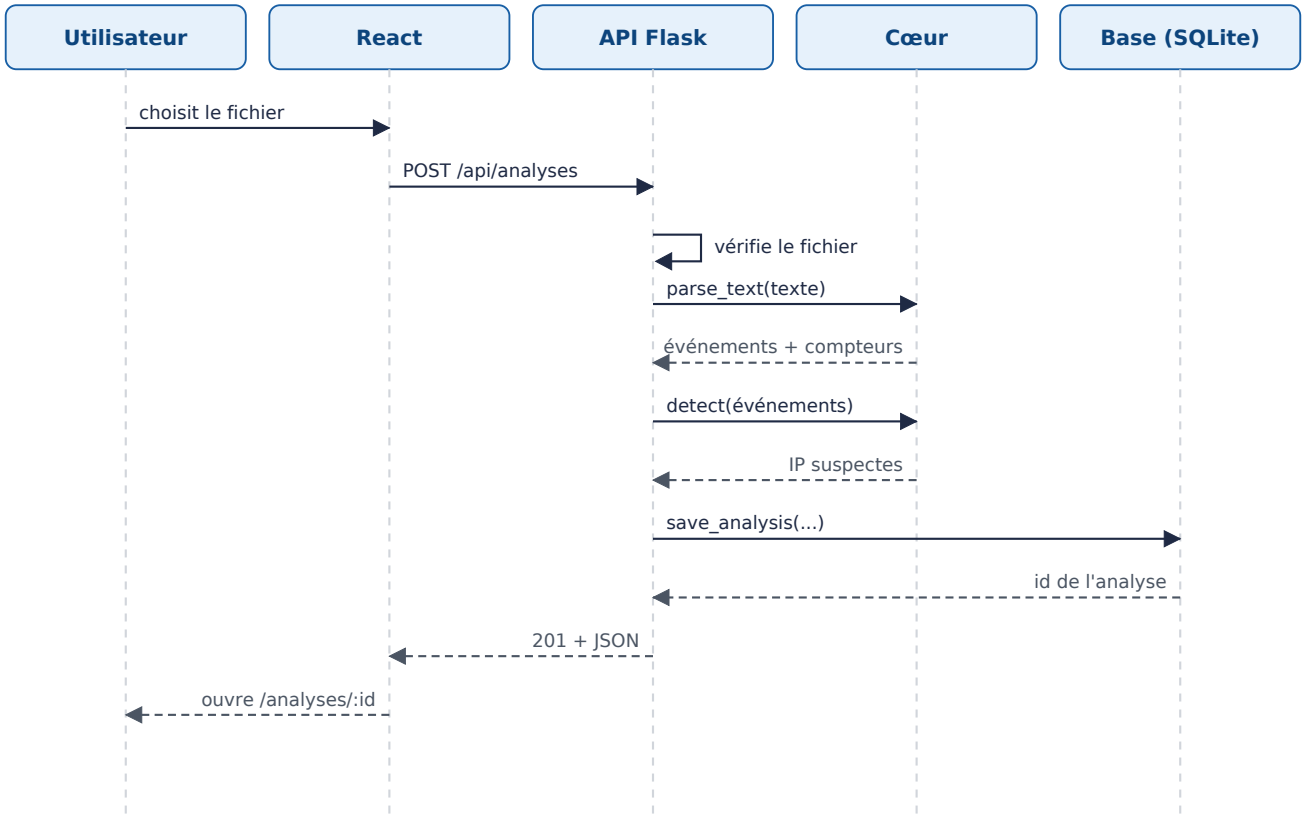
Un diagramme de séquence montre, dans l'ordre, les messages échangés entre les parties du système. Flèche pleine = appel ; flèche en pointillés = réponse.

## 11.1 Se connecter



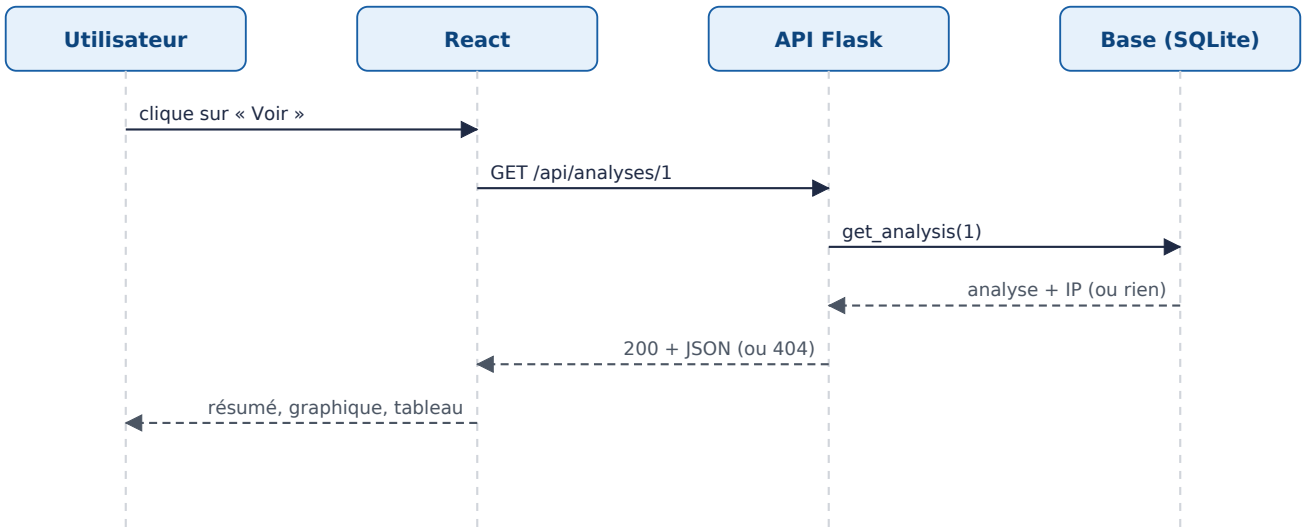

*Si le nom ou le mot de passe est faux, l'API répond 401 avec le même message dans les deux cas.*

## 11.2 Envoyer et analyser un fichier




*Si le fichier est refusé, l'API répond directement 400 ou 413 et React affiche le message d'erreur.*

### 11.3 Consulter une analyse




*La suppression suit le même chemin avec DELETE : la base supprime l'analyse et ses IP, l'API répond 204, puis React recharge l'historique et les récidivistes.*

# 12. Logique de détection

## 12.1 Les lignes à lire

Format ciblé : le format classique d'Ubuntu (`Mois Jour HH:MM:SS`), qui est aussi celui du dataset public OpenSSH de Loghub. Seules 2 familles de lignes t'intéressent :

```
Oct 12 14:03:21 srv sshd[1234]: Failed password for invalid user admin from 203.0.113.45 port 51122 ssh2
Oct 12 14:05:02 srv sshd[1301]: Accepted password for anil from 203.0.113.45 port 51290 ssh2
```

- `Failed password` (y compris « for invalid user ») → **échec**.
- `Accepted password` ou `Accepted publickey` → **succès**.
- Tout le reste → ligne ignorée, mais comptée (le résumé l'affiche).

De chaque ligne utile, tu extrais 4 informations : l'heure, le type (échec ou succès), le compte visé et l'IP. Astuce : l'IP est toujours le mot entre `from` et `port`, ce qui marche aussi pour l'IPv6.

Tu ignores volontairement la ligne `Invalid user admin from ...` : elle décrit la même tentative que `Failed password for invalid user admin`. La compter aussi doublerait les échecs.

## 12.2 Les 3 règles

| Règle | Condition (seuils dans config.py) | Gravité |
|---|---|---|
| **R1** — Attaque persistante | Au moins 20 échecs au total dans le fichier, même étalés dans le temps | **Moyenne** |
| **R2** — Force brute rapide | Au moins 5 échecs en 60 secondes ou moins | **Élevée** |
| **R3** — Compromission possible | Une connexion réussie après au moins 5 échecs de la même IP | **Critique** |

Une IP qui déclenche plusieurs règles garde la gravité la plus haute. Toutes les règles déclenchées sont enregistrées dans `rules`.

### 12.3 Indices pour coder les règles toi-même

- **R2 :** garde, pour chaque IP, la liste de ses heures d'échec dans l'ordre. Tu peux savoir si 5 échecs tiennent en 60 secondes en comparant seulement 2 cases de cette liste. Lesquelles ?
- **R3 :** au moment où tu lis une ligne `Accepted`, quelle information sur cette IP dois-tu déjà avoir en mémoire pour décider ?
- **Comptes les plus visés :** regarde `Counter` dans le module `collections`, et sa méthode `most_common()`.
- **Lecture de la date :** la ligne ne contient pas l'année. Si tu convertis `Feb 29 10:00:00` sans année, Python utilise l'année 1900, qui n'est pas bissextile, et la conversion plante. Ajoute une année bissextile fixe (par exemple 2000) avant de convertir.

### 12.4 Limites connues (à écrire dans ton rapport)

- Le format classique n'indique pas l'année. Ce n'est pas gênant : les règles comparent des écarts de temps, pas des dates (seul le passage du 31 décembre au 1er janvier peut fausser un écart).
- Un seul format de log est supporté (Ubuntu classique).
- Une attaque très lente, sous le seuil de R1, reste invisible.
- Plusieurs personnes derrière une même IP (box, entreprise) sont vues comme une seule source.

## 13. Sécurité et conformité

Un outil de sécurité doit être sûr lui-même. C'est un point que le jury aime creuser.

| Menace | Protection dans SentinelSSH |
|---|---|
| **Injection SQL** : du texte malveillant modifie une requête SQL | Requêtes paramétrées avec `?` partout. Jamais de f-string ni de concaténation dans du SQL. |
| **XSS** : du code injecté s'exécute dans la page web | Échappement automatique de React. Jamais de `dangerouslySetInnerHTML`. Le nom d'utilisateur tenté vient de l'attaquant. |
| **Fichier énorme ou piégé** | `MAX_CONTENT_LENGTH` à 5 Mo, extension vérifiée, décodage avec `errors="replace"`. Le fichier brut n'est jamais écrit sur le disque. |
| **Accès non autorisé** aux analyses | Connexion obligatoire (401 sinon). Mot de passe haché avec Werkzeug, jamais stocké en clair. Session dans un cookie signé par `SECRET_KEY` : HttpOnly (activé par défaut dans Flask) et SameSite=Lax (à régler avec `SESSION_COOKIE_SAMESITE`). `SECRET_KEY` lue dans l'environnement, jamais commitée. |
| **Énumération des comptes** | Même message « Identifiants incorrects » que le nom ou le mot de passe soit faux. |
| **Mot de passe faible** | La commande `create-admin` refuse les mots de passe de moins de 12 caractères. Bonus : 1 minute d'attente après 5 échecs de connexion. |
| **Requête forgée depuis un autre site** (CSRF) | Cookie SameSite=Lax : il n'est pas envoyé par les formulaires d'autres sites. L'API n'accepte que du JSON ou des fichiers envoyés par le front. |
| **Exposition sur le réseau** | L'application écoute sur `127.0.0.1` : accessible uniquement depuis la machine. Une exposition sur Internet demanderait HTTPS (hors périmètre). |
| **Fuite d'informations internes** | Pas de mode debug pendant la démo : il affiche des détails internes et une console de débogage. |
| **Blocage d'une mauvaise IP** | L'outil propose, l'admin décide. Aucune commande n'est exécutée : l'application n'a pas besoin des droits root. |

| Menace | Protection dans SentinelSSH |
|---|---|
| **Dépendances vulnérables** | Versions figées (`package-lock.json`, `requirements.txt`), vérification avec `npm audit`. |

### Conformité RGPD — ton atout « GRC »

> • Une **adresse IP est une donnée personnelle** au sens du RGPD.
>
> • **Base légale :** l'intérêt légitime, car la sécurité des réseaux est explicitement citée par le considérant 49 du RGPD.
>
> • **Minimisation :** seules les IP suspectes et des compteurs sont stockés, jamais le log complet.
>
> • **Droit à l'effacement :** la route `DELETE /api/analyses/<id>` supprime une analyse et ses IP.
>
> • **Durée de conservation :** à fixer et à écrire dans le README (par exemple 30 jours).
>
> • **Pas de données réelles sur GitHub :** ni base, ni vrais logs dans le dépôt.
>
> • **Information des personnes :** la page « Confidentialité » (accessible sans connexion) décrit les données traitées, la finalité, la durée de conservation et les droits. C'est la partie « mentions légales RGPD » attendue du développeur.

# 14. Stratégie de tests et qualité

## 14.1 Tests automatiques du back-end (pytest)

| Fichier | Cas testés |
|---|---|
| `test_parser.py` | Ligne `Failed` → événement correct · ligne « invalid user » → bon compte · ligne `Accepted` → succès · ligne sans rapport → `None` · IP en IPv6 |
| `test_detector.py` | 5 échecs en 50 s → élevée · 5 échecs en 5 min → pas élevée · 20 échecs étalés → moyenne · 5 échecs puis un succès → critique · 1 succès seul → rien · plusieurs règles → gravité la plus haute |
| `test_api.py` | Sans session → 401 · mauvais mot de passe → 401 · connexion → 200 puis accès autorisé · déconnexion → 401 ensuite · POST valide → 201 · sans fichier → 400 · mauvaise extension → 400 · GET id inconnu → 404 · DELETE puis GET → 404 · 2 analyses du même fichier → IP dans les récidivistes |

- **Base de test séparée :** la fixture `tmp_path` de pytest et un `DB_PATH` configurable évitent de toucher à ta vraie base.
- **Couverture :** `pytest --cov=sentinel` mesure le pourcentage de code exécuté par les tests. Objectif : au moins 80 %.
- **Style :** `pycodestyle` vérifie le style du code Python, comme dans les projets Holberton.
- **Tests manuels de l'API :** une collection Postman (ou des commandes curl) des 8 routes, rejouée avant de brancher le front.
- **Robustesse :** le dépôt GitHub `logpai/loghub` contient un échantillon de 2 000 lignes de logs OpenSSH (parfait pour les premiers essais) et renvoie vers la version complète : 655 146 lignes, environ 70 Mo. Elle dépasse la limite d'envoi de 5 Mo : on la teste avec `cli.py` et on note le temps d'analyse dans le README. Ces logs sont réels : ne les commite pas.

- **Intégration continue :** un workflow GitHub Actions lance pycodestyle, pytest, ESLint et la construction du front à chaque push et Pull Request vers `develop` ou `main`. Une croix rouge sur GitHub = quelque chose est cassé.

### 14.2 Qualité du front-end

- **ESLint** (fourni par le modèle Vite) : `npm run lint` doit passer sans erreur.
- **Checklist manuelle** ci-dessous, à dérouler avant chaque fusion importante et avant la démo.

| # | Scénario | Résultat attendu |
|---|---|---|
| 1 | Envoyer `demo.log` | Les 3 niveaux de gravité apparaissent |
| 2 | Envoyer un fichier `.exe` | Message « Extension non autorisée » |
| 3 | Envoyer un fichier de plus de 5 Mo | Message « Fichier trop volumineux » |
| 4 | Envoyer un fichier .log vide | Analyse créée, « Aucune IP suspecte » |
| 5 | Log contenant l'utilisateur `<script>alert(1)</script>` | Affiché comme du texte, aucune alerte ne s'ouvre |
| 6 | Supprimer une analyse | Elle disparaît de l'historique après confirmation |
| 7 | Envoyer 2 fois le même fichier | Les IP apparaissent dans les récidivistes |
| 8 | Cliquer sur « Copier » | « Copié ! » s'affiche, la commande est dans le presse-papiers |
| 9 | Recharger la page de détail | La page s'affiche toujours |
| 10 | Arrêter Flask puis recharger l'accueil | Message d'erreur clair, pas de page blanche |
| 11 | Ouvrir l'accueil sans être connecté | Redirection vers la page de connexion |
| 12 | Se connecter avec un mauvais mot de passe | Message « Identifiants incorrects » |
| 13 | Afficher chaque page en mode mobile (outils du navigateur) | Tout reste lisible, la page ne déborde pas |

### 14.3 Définition de « terminé »

- Le code est fusionné dans `develop` via une Pull Request.
- Les tests existent et passent (CI verte).
- La checklist concernée est validée.
- L'issue GitHub est fermée et le README est à jour si besoin.

## 15. Gestion du code source (Git)

- **Un seul dépôt** GitHub `sentinelssh` avec les dossiers `backend/` et `frontend/`.
- **Branches :** `main` = version stable, toujours présentable, protégée ; `develop` = intégration des fonctionnalités terminées ; une branche par tâche créée depuis `develop` : `feature/parser`, `feature/api-auth`, `fix/upload-413`… À chaque fin de sprint, `develop` est fusionnée dans `main`.
- **Pull Requests :** même seul, chaque branche revient dans `develop` par une PR avec une checklist (tests, style, aucun secret, documentation). Tu relis ton propre code et la CI vérifie les tests.
- **Commits :** petits et fréquents, au format *Conventional Commits* : `feat:`, `fix:`, `test:`, `docs:`, `refactor:`. Exemple : `feat: add R2 fast brute-force rule`.

- **Suivi :** une issue par user story ou tâche, dans le kanban GitHub Projects (À faire / En cours / Terminé).
- **Versions :** un tag par fin de sprint (`v0.1.0` à `v0.3.0`), puis `v1.0.0` au gel (semaine 4) : c'est la version présentée au demo day.
- **.gitignore :** `venv/`, `__pycache__/`, `node_modules/`, `dist/`, `*.db`, `.env`.

# 16. Installation et lancement

## 16.1 En développement

Prérequis : Python 3.10 ou plus récent, Node.js en version LTS, Git. En développement, l'application se lance dans 2 terminaux.

```
# Terminal 1 : back-end
cd backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env         # puis y coller une SECRET_KEY générée avec :
                             # python -c "import secrets; print(secrets.token_hex(32))"
flask --app app create-admin # une seule fois : crée le compte administrateur
flask --app app run          # API sur http://127.0.0.1:5000

# Terminal 2 : front-end
cd frontend
npm install
npm run dev                  # application sur http://localhost:5173

# Tests et qualité
cd backend && pytest --cov=sentinel
cd frontend && npm run lint

# Mode terminal (secours)
cd backend && python cli.py ../demo/demo.log
```

- Le mode debug de Flask (`--debug`) recharge le serveur à chaque modification : pratique en développement, mais jamais pendant la démo.
- Le README reprend exactement ces commandes. Test final : cloner le dépôt dans un dossier vide et tout relancer en suivant le README.

## 16.2 Mise en production locale (déploiement documenté)

Pour la démo, on n'utilise plus les 2 serveurs de développement. Le front est « construit » en fichiers statiques, Flask les sert lui-même, et Gunicorn lance le tout en une seule commande : moins de pièces qui peuvent casser le jour J.

```
# 1. Construire le front (crée frontend/dist)
cd frontend && npm run build

# 2. Préparer le back (une seule fois)
cd ../backend && source venv/bin/activate
flask --app app create-admin      # si le compte n'existe pas encore

# 3. Lancer en mode production
gunicorn -b 127.0.0.1:8000 app:app
# -> application complète sur http://127.0.0.1:8000
```

- Dans `app.py`, Flask sert les fichiers de `frontend/dist` et renvoie `index.html` pour toute adresse qui ne commence pas par `/api`. C'est ce qui permet à React Router de gérer `/login` ou `/analyses/3`

après un rechargement.

- La `SECRET_KEY` est lue dans `.env` grâce à python-dotenv. Appelle `load_dotenv()` au début de `app.py` : la commande `flask` charge `.env` toute seule, mais Gunicorn non. Cette clé ne doit jamais apparaître dans le code ni sur GitHub.
- Le README contient une section « Déploiement » qui reprend ces étapes, les prérequis et une vérification finale (se connecter, lancer une analyse).
- **Pré-production :** le même montage, lancé sur ta machine avec une base de test et `demo.log`, sert à valider une version avant de la fusionner dans `main`. La production est la machine de démo, avec le vrai compte administrateur.
- Pour une vraie mise en ligne (hors périmètre) : serveur Linux, Nginx devant Gunicorn, HTTPS avec Let's Encrypt, pare-feu.

# 17. Objectifs SMART

Spécifique, Mesurable, Atteignable, Réaliste, Temporel : chaque objectif a une preuve et une échéance.

| Objectif | Mesure (la preuve) | Échéance |
|---|---|---|
| **O1 — Détection** | Les 3 règles donnent le résultat attendu sur 100 % des fichiers de `tests/samples` (pytest au vert) | Fin S1 |
| **O2 — API et base** | Les 8 routes renvoient les données et les codes HTTP prévus au §10 (401 sans connexion), avec persistance dans SQLite (`test_api.py` au vert) | Fin S2 |
| **O3 — Interface React** | Une fois connecté, on passe de l'envoi du fichier à l'affichage du tableau et du graphique en 3 clics maximum ; les 13 scénarios du §14.2 sont validés | Fin S3 |
| **O4 — Robustesse** | Les 655 146 lignes du dataset OpenSSH de Loghub sont analysées sans plantage ; le temps d'analyse est noté dans le README | Fin S3 |
| **O5 — Qualité et déploiement** | Couverture des tests d'au moins 80 % sur `sentinel/`, CI verte, ESLint sans erreur ; application lancée en production locale en suivant uniquement le README | Fin S4 |

# 18. Risques et parades

| Risque | Conséquence | Parade |
|---|---|---|
| Découverte de React | Retard sur le front | Tutoriel officiel de react.dev en parallèle dès la semaine 1 · API finie et testée avant le front · composants simples, sans bibliothèque d'état |
| Problème de communication front ↔ back (CORS) | Le front ne reçoit rien | Proxy Vite (§6.4) · tester l'API seule avec curl avant de brancher React |
| Clés étrangères SQLite désactivées par défaut | La cascade ne marche pas | `PRAGMA foreign_keys = ON` à chaque connexion + un test dédié |
| Formats de logs variables | Lignes mal lues | Un seul format ciblé · lignes ignorées comptées et affichées |
| Seuils mal calibrés | Fausses alertes ou attaques manquées | Seuils dans `config.py`, validés sur les fichiers de test |
| Authentification mal implémentée | Données exposées | Fonctions éprouvées de Werkzeug et sessions Flask, rien de « fait maison » · tests 401 dédiés |
| Semaine 2 chargée (base + API + connexion) | Retard en cascade | Un seul compte, pas d'inscription · limitation des tentatives en bonus seulement |
| Code généré par IA mal compris | Échec aux questions du jury | Cœur et SQL écrits soi-même · aucune ligne commitée sans savoir l'expliquer |
| Retard général | Fonctionnalités inachevées | Planning par couches + ordre de coupe (§19) |

# 19. Planning sur 4 semaines

On construit de l'intérieur vers l'extérieur : d'abord le cœur, puis la base et l'API, puis le front. À chaque fin de semaine, tu as quelque chose qui marche et que tu peux montrer.

| Semaine | Travail | En parallèle | Terminé quand… |
|---|---|---|---|
| **S1** — Le cœur | `parser.py`, `detector.py`, `config.py`, `cli.py` + tests (F2, F3, F10) | Tutoriel React officiel (react.dev), environ 1 h par jour | `python cli.py demo/demo.log` affiche les bonnes IP, pytest au vert |
| **S2** — Base + API | `schema.sql`, `database.py`, connexion (3 routes, `create-admin`), les 5 routes d'analyse, erreurs, `test_api.py`, CI (F4, F7, F8, F9, F11) | Créer le projet Vite et afficher l'historique depuis l'API | Toutes les routes marchent avec curl, tests et CI au vert |
| **S3** — Front React | Les 4 pages et les 10 composants, `api.js`, CSS responsive (F1, F5, F6, F7, F8, F9, F11, F12, F13) | Dérouler la checklist au fur et à mesure | Parcours complet dans le navigateur avec `demo.log` |
| **S4** — Gel | Plus aucune nouvelle fonctionnalité : mise en production (§16.2), corrections, test Loghub, README, rapport, tag `v1.0.0` | Répétitions de la démo | 3 démos d'affilée sans bug |

### Si tu prends du retard

- **Coupe dans cet ordre :** 1) les bonus, 2) le graphique, 3) les récidivistes.

- **Ne coupe jamais :** la détection, la base, l'API, la connexion, le tableau des résultats, la page de confidentialité et les tests.

Ce planning suppose de travailler sur le projet presque tous les jours. Si tu as moins de temps, applique l'ordre de coupe dès la fin de la semaine 2, sans attendre.

# 20. Préparer la soutenance

## 20.1 Scénario de démo (environ 5 minutes)

1. Présenter le problème en 30 secondes : montrer un vrai `auth.log` qui défile.
2. Se connecter : l'application est protégée, comme tout outil de sécurité devrait l'être.
3. Envoyer `demo.log` : le résumé, le graphique et le tableau apparaissent.
4. Montrer les 3 niveaux : une IP qui insiste lentement (moyenne), une rafale (élevée), puis une IP qui finit par réussir (critique). C'est le moment fort.
5. Copier la commande de blocage et expliquer pourquoi la décision reste humaine.
6. Envoyer un 2e fichier : la page d'accueil fait apparaître une IP récidiviste.
7. Supprimer une analyse et parler du RGPD.
8. Montrer rapidement les tests et la CI verte sur GitHub.

Écris `demo.log` toi-même avec des adresses réservées à la documentation (`203.0.113.x`, `198.51.100.x`, RFC 5737) : tu ne montres aucune vraie IP. Lance la version de production (§16.2), un seul serveur. Si le web plante, rejoue la démo avec `cli.py`.

## 20.2 Questions probables du jury

| Question | Où trouver ta réponse |
|---|---|
| Pourquoi ne pas utiliser fail2ban ? | §1.4 |
| Pourquoi Flask, React et SQLite ? | §5 |
| Où sont les compétences du référentiel dans ton projet ? | §2.5 |
| Comment stockes-tu les mots de passe ? Hachage ou chiffrement ? | §9.1 et §13 |
| Comment déploies-tu ton application ? | §16.2 |
| Ton application fonctionne-t-elle sur mobile ? Est-elle accessible ? | §8.5 |
| Comment ton front parle-t-il à ton back ? C'est quoi CORS ? | §6.4 |
| Qu'est-ce qu'une API REST ? Pourquoi 201 et pas 200 ? | §10 |
| C'est quoi un composant, des props, un state ? | §8 |
| Comment empêches-tu une injection SQL ? | §13 |
| Et si un attaquant met du code dans son nom d'utilisateur ? | §8.4 et §13 |
| Explique ta règle des 60 secondes. | §12.2 et §12.3 |
| Pourquoi l'outil ne bloque-t-il pas lui-même les IP ? | §2.3 et §13 |
| Comment as-tu testé ton projet ? | §14 |
| Comment as-tu organisé ton code et ton travail ? | §15 et §19 |
| Quelles sont les limites de ton outil, et la suite ? | §2.4 et §12.4 |
| Comment respectes-tu le RGPD ? | §13 |

## 20.3 Si tu codes avec l'IA

> Au jury, on te demandera d'expliquer ton code. Écris toi-même `parser.py`, `detector.py` et `database.py` : c'est le cœur, et c'est là que viendront les questions. L'IA peut t'aider davantage sur le CSS et la mise en forme React. **Règle d'or :** ne commite jamais une ligne que tu ne saurais pas expliquer à voix haute.

# Annexe — Lexique

| Terme | Définition simple |
|---|---|
| API | Ensemble de routes qu'un programme expose pour que d'autres programmes l'utilisent. |
| REST | Façon d'organiser une API autour de ressources (ici, les analyses) et des méthodes HTTP (GET, POST, DELETE). |
| JSON | Format texte pour échanger des données structurées, lisible par Python et JavaScript. |
| Back-end / Front-end | Back-end : la partie serveur (logique, API, base). Front-end : la partie visible dans le navigateur. |
| React | Bibliothèque JavaScript pour construire une interface à partir de composants. |
| Composant | Fonction React qui renvoie un morceau d'interface, réutilisable. |
| Props | Données qu'un composant reçoit de son parent. |
| State | Données qu'un composant garde ; quand elles changent, React réaffiche le composant. |
| Hook | Fonction React qui commence par « use » : `useState` (state), `useEffect` (code lancé après l'affichage, par exemple un appel à l'API). |
| Vite | Outil qui crée, lance et construit un projet front-end. |
| npm | Gestionnaire de paquets de Node.js : installe les bibliothèques JavaScript. |
| CORS | Règle du navigateur qui bloque par défaut les requêtes vers une autre origine (autre domaine ou autre port). |
| Proxy | Intermédiaire qui reçoit une requête et la transmet à un autre serveur. |
| Base de données | Endroit où les données sont rangées de façon durable, en tables. |
| Clé primaire / étrangère | Primaire : identifie chaque ligne. Étrangère : pointe vers la clé primaire d'une autre table. |
| ON DELETE CASCADE | Règle qui supprime automatiquement les lignes liées quand on supprime la ligne parente. |
| Force brute | Attaque qui essaie un très grand nombre de mots de passe jusqu'à trouver le bon. |
| Injection SQL | Attaque où du texte fourni par l'utilisateur modifie une requête SQL. |
| Requête paramétrée | Requête où les valeurs sont passées à part (avec `?`), jamais collées dans le texte SQL. |
| XSS | Attaque où du code malveillant est affiché, puis exécuté, dans une page web. |
| User story | Besoin décrit du point de vue de l'utilisateur : « En tant que…, je veux…, afin de… ». |
| Pull Request (PR) | Demande de fusion d'une branche dans `main`, avec relecture et tests. |
| Couverture de tests | Pourcentage du code exécuté pendant les tests. |
| CI | Intégration continue : les tests se lancent automatiquement à chaque envoi de code. |
| MVP | Plus petite version du produit qui fonctionne vraiment et qu'on peut montrer. |
| RGPD | Règlement européen sur la protection des données personnelles. |
| Hachage | Transformation à sens unique d'un mot de passe en empreinte : on peut vérifier un mot de passe, jamais le retrouver. |

| Terme | Définition simple |
|---|---|
| Session / cookie | Petit fichier que le serveur fait garder au navigateur pour le reconnaître après la connexion. |
| 401 | Code HTTP « non authentifié » : il faut se connecter pour accéder à cette route. |
| Responsive | Interface qui s'adapte à la taille de l'écran (ordinateur, tablette, téléphone). |
| Gunicorn | Serveur de production pour les applications web Python comme Flask. |
| Build (construction) | Étape qui transforme le code React en fichiers HTML, CSS et JavaScript optimisés, prêts à être servis. |